STAT 405 Final Project

Alec Neme (84761030), Mikayla Liang (87701603)

2026-03-15

Abstract

Investor confidence is fundamentally tied to market stability, as sudden transitions into high-volatility regimes expose investors to significant risk. This study explores the use of Bayesian Inference frameworks in detecting and predicting these volatile periods. Using historical data from Yahoo Finance spanning January 1998 to the present, the comparative models identify latent shifts in market behaviour driven by rapid economic change. The results demonstrate that ...

Introduction

This timeframe was selected to encompass three major structural market shocks: the Dot-com Bubble, the 2008 Financial Crisis, and the 2020 COVID-19 crash. Our primary analysis will focus on the S&P 500 Index. To validate the accuracy of our approach, we may utilize the CBOE Volatility Index (VIX), allowing us to compare our model's estimated latent volatility against Wall Street's implied volatility. Additionally, assets such as sector-specific ETFs (XLK, XLF) and a safe-haven treasury bond (TLT) will be incorporated to test if the model's regime-switching logic generalizes reliably across different market segments, as well as sector-specific insight of each crisis. To reduce the computational burden while preserving essential volatility clustering, the daily data will be mutated to only include the final trading day of each week to establish consistent weekly boundaries. Finally, these weekly prices will be transformed into weekly log returns, calculated as $Y_t = \ln(P_t/P_{t-1})$. This transformation is critical, as it ensures a stationary distribution of the time series which facilitates its modelling.

Which ticker is each index?

- ^GSPC (S&P 500 Index): Top 500 stocks in the U.S. This is the observation that we will most likely use.
- ^VIX (CBOE Volatility Index): Market volatility measure that we can use for model validation.
- XLK (Technology Select Sector SPDR Fund): An ETF containing all the tech stocks in the S&P 500. Maybe use for dot com crash?
- XLF (Financial Select Sector SPDR Fund): An ETF containing all the financial stocks in the S&P 500 (banks, insurance, etc.). Check for 2008 Financial Crisis?
- TLT (iShares 20+ Year Treasury Bond ETF): This tracks long-term US government bonds. "Safe haven" asset that normally behaves opposite to stocks. Does the model still work with this?

We may also choose to include some other indexes.

```
project_tickers <- c("^GSPC", "^VIX", "XLK", "XLF", "TLT")
market_data <- tq_get(project_tickers,
                      get  = "stock.prices",
                      from = "1998-01-01")

weekly_returns <- market_data %>%
```

```r
  mutate(week = floor_date(date, "week")) %>%
  group_by(symbol, week) %>%
  arrange(date) %>%
  slice_tail(n = 1) %>%
  ungroup()


weekly_log_returns <- weekly_returns %>%
  group_by(symbol) %>%
  arrange(date) %>%
  mutate(
    previous_price = lag(adjusted),
    log_return = log(adjusted / previous_price)
  ) %>%
  filter(!is.na(log_return)) %>%
  select(symbol, date, adjusted, log_return)

head(weekly_log_returns)
```

```
## # A tibble: 6 x 4
## # Groups:   symbol [2]
##   symbol date       adjusted log_return
##   <chr>  <date>        <dbl>      <dbl>
## 1 ^GSPC  1998-01-09    928.    -0.0498
## 2 ^VIX   1998-01-09     28.7    0.203
## 3 ^GSPC  1998-01-16    962.     0.0358
## 4 ^VIX   1998-01-16     21.6   -0.282
## 5 ^GSPC  1998-01-23    958.    -0.00409
## 6 ^VIX   1998-01-23     23.3    0.0743
```

Our backup dataset uses the 'quantmod' library which also pulls live data from Yahoo Finance. The data has also been adjusted to match the structure of the primary dataset.

```r
library(quantmod)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats 1.0.0     v stringr 1.5.0
## v purrr   1.0.2     v tibble  3.2.1
## v readr   2.1.4     v tidyr   1.3.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::first()  masks xts::first()
## x dplyr::lag()    masks stats::lag()
## x dplyr::last()   masks xts::last()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
getSymbols(project_tickers, src = "yahoo", from = "1998-01-01")
```

```
## Warning: ^VIX contains missing values. Some functions will not work if objects
## contain missing values in the middle of the series. Consider using na.omit(),
## na.approx(), na.fill(), etc to remove or replace them.
```

```
## [1] "GSPC" "VIX"  "XLK"  "XLF"  "TLT"
```

```r
restructured_data <- do.call(rbind, lapply(project_tickers, function(x) {
```

```r
  clean_name <- gsub("\\^", "", x)
  if(!exists(clean_name)) return(NULL)

  obj <- get(clean_name)

  names(obj) <- c("Open", "High", "Low", "Close", "Volume", "Adjusted")
  df <- data.frame(date = index(obj), coredata(obj))

  df$ticker <- x
  return(df)
}))

restructured_data <- restructured_data %>%
  select(date, ticker, High, Low, Close, Volume, Adjusted)

weekly_returns2 <- restructured_data %>%
  mutate(week = floor_date(date, "week")) %>%
  group_by(ticker, week) %>%
  arrange(date) %>%
  slice_tail(n = 1) %>%
  ungroup()

weekly_log_returns2 <- weekly_returns2 %>%
  group_by(ticker) %>%
  arrange(date) %>%
  mutate(
    previous_price = lag(Adjusted),
    log_return = log(Adjusted / previous_price)
  ) %>%
  filter(!is.na(log_return)) %>%
  select(ticker, date, Adjusted, log_return)

head(weekly_log_returns2)
```

```
## # A tibble: 6 x 4
## # Groups:   ticker [2]
##   ticker date       Adjusted log_return
##   <chr>  <date>        <dbl>      <dbl>
## 1 ^GSPC  1998-01-09    928.    -0.0498
## 2 ^VIX   1998-01-09     28.7    0.203
## 3 ^GSPC  1998-01-16    962.     0.0358
## 4 ^VIX   1998-01-16     21.6   -0.282
## 5 ^GSPC  1998-01-23    958.    -0.00409
## 6 ^VIX   1998-01-23     23.3    0.0743
```

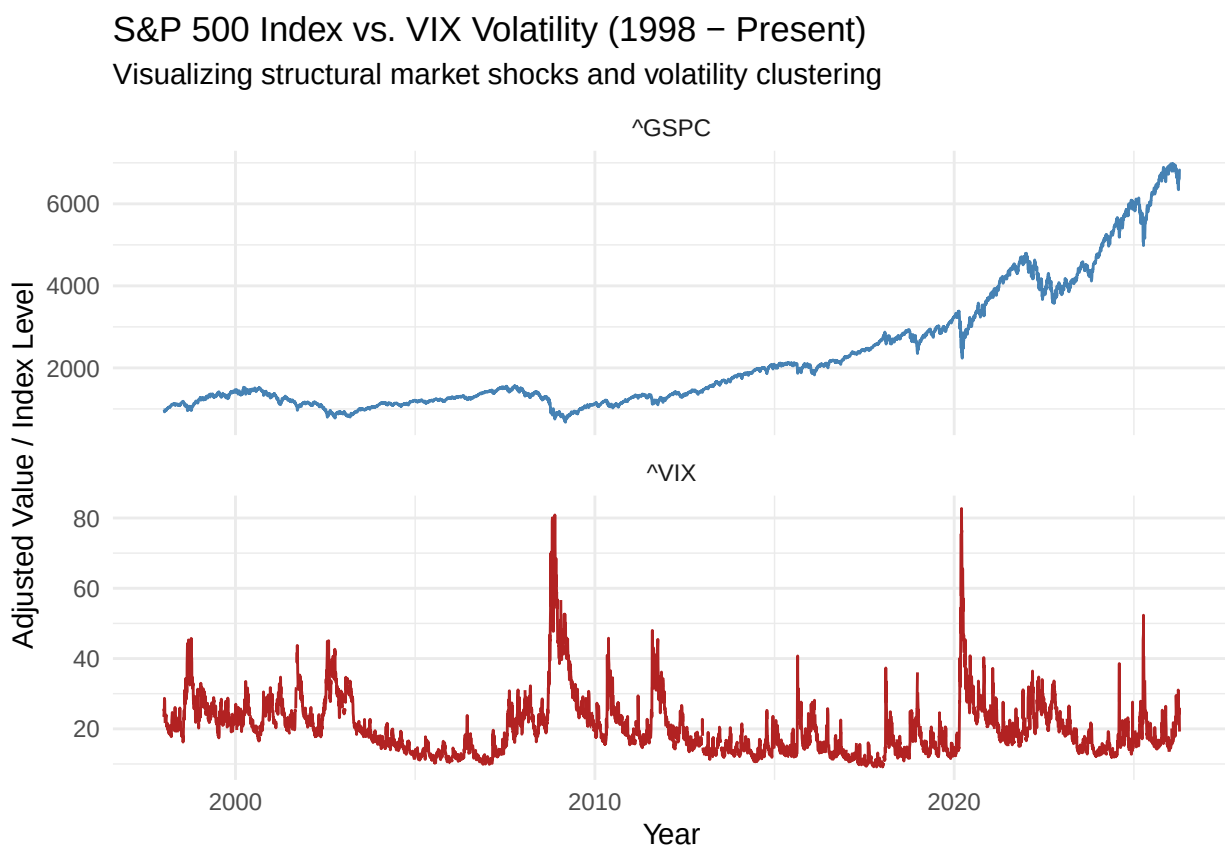
Scientific Questions

1. Between discrete regime shifts (HMM) and continuous volatility evolution (Stochastic Volatility), which framework better explains the market dynamics during the Dot-com bubble, the 2008 Financial Crisis and the COVID-19 crisis?

2. To what extent do the differing acceleration rates of market stress during the Dot-com bubble, the 2008 Financial Crisis, and the COVID-19 crash affect the comparative fit of discrete (HMM) versus continuous (SV) frameworks?

EDA

```r
# Using the market_data object you created earlier
market_data %>%
  filter(symbol %in% c("^GSPC", "^VIX")) %>%
  ggplot(aes(x = date, y = adjusted, color = symbol)) +
  geom_line(show.legend = FALSE) +
  facet_wrap(~symbol, scales = "free_y", ncol = 1) +
  theme_minimal() +
  labs(title = "S&P 500 Index vs. VIX Volatility (1998 - Present)",
       subtitle = "Visualizing structural market shocks and volatility clustering",
       x = "Year",
       y = "Adjusted Value / Index Level") +
  scale_color_manual(values = c("^GSPC" = "steelblue", "^VIX" = "firebrick"))
```



The visualization highlights a clear inverse correlation: significant price contractions in the S&P 500 (^GSPC) correspond directly to sharp spikes in the VIX. These synchronized movements clearly mark the three major structural shocks identified in our proposal. The abrupt, vertical nature of these volatility surges visually mimics discrete state transitions, reinforcing our decision to utilize a Hidden Markov Model (HMM).

Furthermore, the raw S&P 500 series exhibits a distinct upward trend over the 25-year period, indicating it is non-stationary. By transforming these prices into weekly log returns, $Y_t = \ln(P_t/P_{t-1})$, we remove this drift and stabilize the variance. This transformation is mathematically critical; it ensures our HMM operates on a stationary distribution, which is a necessary condition for the model to reliably infer the "Calm" and "Crisis" latent states over the long term.

Hidden Markov Model

The Likelihood

Let Y_t represent the weekly log return of the index at time t, and let $X_t \in \{1, 2\}$. State 1 represents a "Calm/Normal" market regime, and State 2 represents a "Crisis" regime.

$$Y_t \mid X_t = i \sim \mathcal{N}(\mu_i, \sigma_i^2)$$

Where:

$$\sigma_1 \sim Exponential(1)$$
$$\sigma_{diff} \sim Exponential(1)$$
$$\sigma_2 = \sigma_1 + \sigma_{diff}$$

The Latent State Dynamics (The Regime Switching)

The sequence of unobserved market states follows a first-order Markov process. The probability distribution of tomorrow's state, X_t, is explicitly conditioned on today's state, X_{t-1}.

Latent States

$$X_t \mid X_{t-1} = 1 \sim \text{Categorical}(P_{11}, 1 - P_{11})$$
$$X_t \mid X_{t-1} = 2 \sim \text{Categorical}(1 - P_{22}, P_{22})$$

Transition Probabilities

$$P = \begin{pmatrix} P_{11} & 1 - P_{11} \\ 1 - P_{22} & P_{22} \end{pmatrix}$$

where,

$$P_{11} \sim \text{Beta}(1, 1)$$
$$P_{22} \sim \text{Beta}(1, 1)$$

The Marginalized Joint Likelihood

Because Stan's Hamiltonian Monte Carlo (HMC) algorithm relies on continuous gradient calculations, it cannot directly sample discrete latent parameters. Consequently, we must apply Rao-Blackwellization by computing the marginal posterior, effectively summing out the discrete state sequence $X_{1:T}$.

Over the whole dataset, the brute-force marginalized joint likelihood requires summing over the two states $\{1, 2\}$ for all T weeks:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \sum_{x_2=1}^{2} \cdots \sum_{x_T=1}^{2} p(Y_{1:T}, X_{1:T} = x_{1:T}|\theta, P)$$

By factoring the joint distribution into the initial state probabilities π, the transition matrix P, and the state-dependent emission likelihoods, we obtain the fully expanded equation:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \cdots \sum_{x_T=1}^{2} \left[\pi_{x_1} p(Y_1|\mu_{x_1}, \sigma_{x_1}^2) \prod_{t=2}^{T} P_{x_{t-1}, x_t} p(Y_t|\mu_{x_t}, \sigma_{x_t}^2) \right]$$

This exhaustive summation involves 2^T unique possible paths. Computing this $\mathcal{O}(2^T)$ sum at every MCMC step is computationally expensive.

Rao-Blackwellization via the Forward Algorithm

To resolve this computational bottleneck, we utilize the Forward Algorithm. The Forward Algorithm is a dynamic programming approach that relies on the conditional independence of the HMM. Instead of calculating the probability of every single distinct path from week 1 to week T independently, the algorithm calculates the probability sequentially and caches the intermediate results.

Because the Markov property dictates that the future state depends only on the present state, we do not need to track how the market arrived at a certain state, only the accumulated probability mass of being there. We define a forward variable, $\alpha_t(j)$, representing the joint probability of observing the sequence of returns up to time t and currently being in state j:

$$\alpha_t(j) = p(Y_{1:t}, X_t = j|\theta, P)$$

The algorithm computes this dynamically in three steps:

1. Base Case ($t = 1$):

$$\alpha_1(j) = \pi_j p(Y_1|\mu_j, \sigma_j^2) \quad \text{for } j \in \{1, 2\}$$

2. Recursive Step ($t = 2, \ldots, T$): By summing the cached probabilities from the previous week ($\alpha_{t-1}(i)$) and multiplying by the transition probability P_{ij}, we find the total probability of arriving at state j. We then multiply by the new emission likelihood:

$$\alpha_t(j) = p(Y_t|\mu_j, \sigma_j^2) \sum_{i=1}^{2} \alpha_{t-1}(i) P_{ij}$$

3. Termination (Marginalized Likelihood): At the final week T, the algorithm has compressed the 2^T paths into just two values: $\alpha_T(1)$ and $\alpha_T(2)$, representing the joint probability of the entire dataset ending in State 1 and State 2, respectively. By the Law of Total Probability, since the market must end in one of these two mutually exclusive states, the final marginalized likelihood is simply their sum:

$$L(\theta, P; Y_{1:T}) = p(Y_{1:T}|\theta, P) = \sum_{j=1}^{2} \alpha_T(j)$$

This reduces the time complexity from $\mathcal{O}(2^T)$ to $\mathcal{O}(T)$.

Log-Space Implementation

While the Forward Algorithm solves the theoretical complexity, implementing it directly may cause underflow. Multiplying hundreds of small probability values quickly results in a number smaller than a computer's floating-point limit (e.g., 10^{-308}), causing the software to round the total likelihood to zero.

To maintain numerical stability, Stan evaluates the marginalized posterior strictly in log-space. We define the log-forward variable as $\log \alpha_t(j)$.The base case becomes a simple addition:

$$\log \alpha_1(j) = \log \pi_j + \log p(Y_1|\mu_j, \sigma_j^2)$$

The recursive step requires taking the log of a sum, which Stan handles using the stable log_sum_exp function:

$$\log \alpha_t(j) = \log p(Y_t|\mu_j, \sigma_j^2) + \log \left(\sum_{i=1}^{2} \exp(\log \alpha_{t-1}(i) + \log P_{ij}) \right)$$

Finally, the total marginalized log-likelihood is the log-sum-exp of the final terminal states:

$$\log L(\theta, P; Y_{1:T}) = \log \left(\sum_{j=1}^{2} \exp(\log \alpha_T(j)) \right)$$

Implmentation of HMM

```
# Compile the model
model <- cmdstan_model("/Users/mikaylalynn/Desktop/School/stat405-project/hmm_volatility.stan")

# Prepare data (using your existing weekly_log_returns object)
gspc_returns <- weekly_log_returns %>%
  filter(symbol == "^GSPC") %>%
  arrange(date)

stan_data <- list(
  T = nrow(gspc_returns),
  Y = gspc_returns$log_return
)

# Fit the model
fit <- model$sample(
  data      = stan_data,
  chains    = 4,
  parallel_chains = 4,       # run chains in parallel
  iter_warmup   = 1000,
  iter_sampling = 1000,
  adapt_delta   = 0.95,
  seed      = 405             # reproducibility
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/3g/16zg
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 1
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/3g/16zg
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/3g/16zg
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 3
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/3g/16zg
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
```

```
## Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 54.8 seconds.
## Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 59.8 seconds.
## Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 62.0 seconds.
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 72.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 62.3 seconds.
```

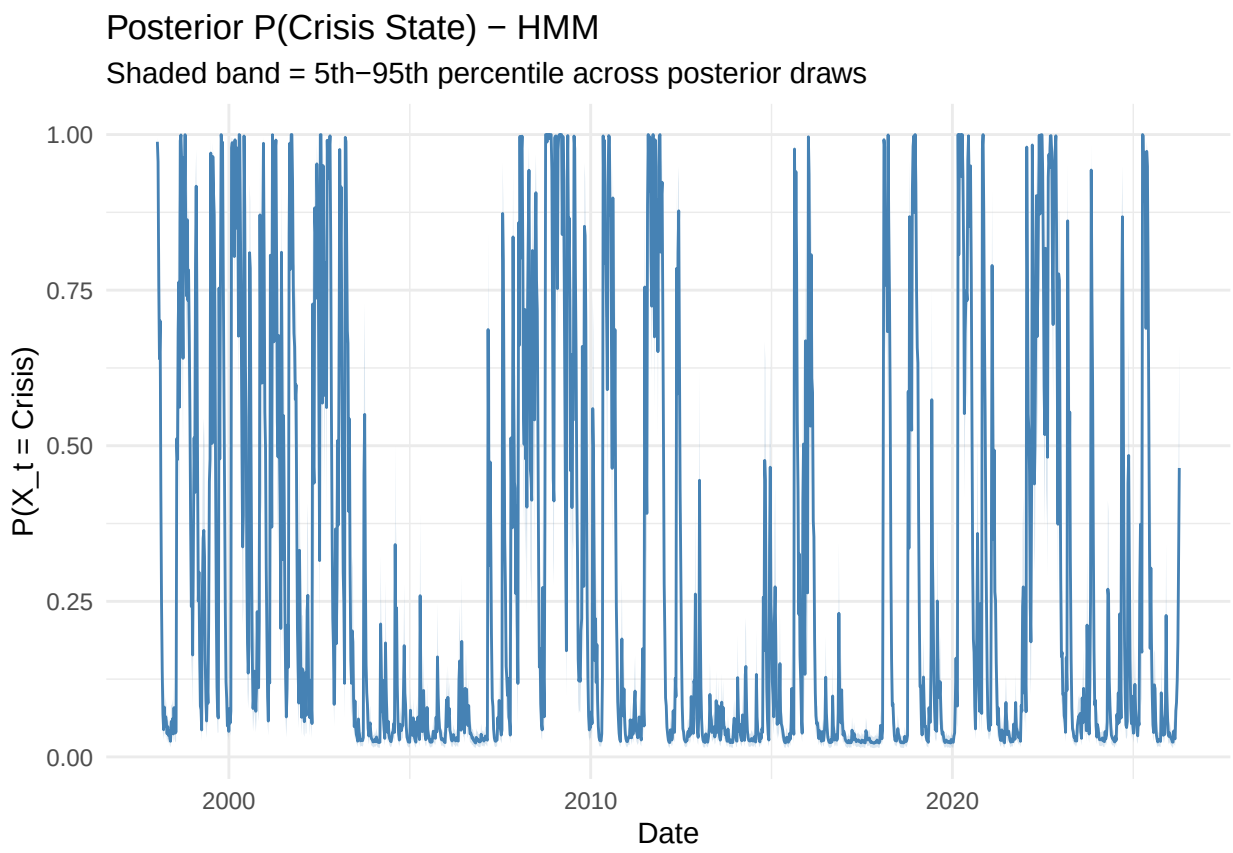
```
## Total execution time: 72.5 seconds.
```

```r
# Summary of key parameters
fit$summary(variables = c("mu", "sigma_1", "sigma_2", "P11", "P22"))
```

```
## # A tibble: 6 x 10
##   variable     mean   median       sd      mad       q5       q95  rhat ess_bulk
##   <chr>       <dbl>    <dbl>    <dbl>    <dbl>    <dbl>     <dbl> <dbl>    <dbl>
## 1 mu[1]     0.00337  0.00337 0.000509 0.000506  0.00252  0.00418   1.00    3686.
## 2 mu[2]    -0.00352 -0.00353 0.00194  0.00195  -0.00669 -0.000431  1.00    5430.
## 3 sigma_1   0.0151   0.0151  0.000543 0.000550  0.0143   0.0160    1.00    2178.
## 4 sigma_2   0.0388   0.0387  0.00186  0.00182   0.0359   0.0420    1.00    2397.
## 5 P11       0.965    0.965   0.00887  0.00870   0.949    0.978     1.00    1972.
## 6 P22       0.915    0.917   0.0217   0.0214    0.877    0.947     1.00    2266.
## # i 1 more variable: ess_tail <dbl>
```

```r
# Extract prob_crisis for plotting
prob_crisis <- fit$summary("prob_crisis") %>%
  mutate(date = gspc_returns$date)
```

```r
# Plot regime probabilities against VIX
prob_crisis %>%
  ggplot(aes(x = date, y = mean)) +
  geom_line(color = "steelblue") +
  geom_ribbon(aes(ymin = q5, ymax = q95), alpha = 0.2, fill = "steelblue") +
  labs(
    title    = "Posterior P(Crisis State) - HMM",
    subtitle = "Shaded band = 5th-95th percentile across posterior draws",
    x = "Date", y = "P(X_t = Crisis)"
  ) +
  theme_minimal()
```


Posterior P(Crisis State) − HMM
Shaded band = 5th−95th percentile across posterior draws
1.00
0.75
0.50
0.25
0.00
P(X_t = Crisis)
2000
2010
2020
Date